МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Вычислительная математика»

Студент гр. 4343 _____ Гизатов Я.Р.

Преподаватель _____ Лисс А. Р.

Санкт-Петербург

2026

Цель работы.

Изучение прямых (метод Гаусса) и итерационных (метод Якоби) алгоритмов решения систем линейных алгебраических уравнений, исследование их точности и устойчивости на матрицах различной обусловленности, сравнение с реализациями в библиотеке Eigen.

Задание.

1. Написать программу для решения СЛАУ методом Гаусса с полным выбором ведущего элемента.
2. Реализовать метод простых итераций (Якоби) для сравнения.
3. Показать корректную работу на больших случайных матрицах, на матрицах Гильберта, на вырожденных и диагонально преобладающих системах.
4. Сравнить полученные решения с эталонным (точным) решением и с результатами библиотеки Eigen.
5. Проанализировать влияние числа обусловленности и нормы матрицы итераций на сходимость.

Метод Гаусса (с полным выбором ведущего элемента)

Модифицированный метод Гаусса с полным выбором ведущего элемента представляет собой усовершенствованный алгоритм решения систем линейных алгебраических уравнений, который значительно повышает численную устойчивость вычислений по сравнению с классическим методом. Основное отличие заключается в стратегии выбора ведущего элемента: если в классическом методе Гаусса ведущим элементом на каждом шаге выбирается диагональный элемент текущей строки, то в модифицированном варианте производится поиск максимального по модулю элемента во всей оставшейся подматрице, включая как строки, так и столбцы. Алгоритм состоит из нескольких этапов. На первом этапе выполняется инициализация векторов перестановок строк и столбцов, которые запоминают произведённые перестановки для последующего восстановления порядка неизвестных. На втором этапе реализуется прямой ход метода. Для каждого шага k от 0 до n-1 сначала осуществляется поиск максимального по модулю элемента в подматрице, ограниченной строками от k до n-1 и столбцами от k до n-1. Найденный элемент становится ведущим. Затем производится перестановка строк: если строка с максимальным элементом не совпадает с текущей строкой k, строки меняются местами, и соответствующая перестановка запоминается в векторе перестановок строк. Аналогично выполняется перестановка столбцов: если столбец с максимальным элементом не совпадает с текущим столбцом k, все элементы в этом столбце переставляются, и перестановка запоминается в векторе перестановок столбцов. После этого для всех строк i от k+1 до n-1 вычисляется множитель как отношение A[i][k] к A[k][k], и из каждой такой строки вычитается ведущая строка, умноженная на этот множитель, что приводит к обнулению элементов ниже диагонали в текущем столбце. На третьем этапе выполняется обратный ход. Начиная с последней строки и поднимаясь вверх, для каждой строки i вычисляется сумма произведений элементов A[i][j] на уже найденные неизвестные y[j] для j от i+1 до n-1,

после чего неизвестное y[i] находится как (A[i][n] - сумма) делённое на диагональный элемент A[i][i]. Полученный вектор y соответствует решению системы с переставленными столбцами. На четвёртом этапе производится восстановление исходного порядка неизвестных. Поскольку в процессе прямого хода столбцы переставлялись, необходимо применить обратные перестановки: для каждого i новое значение x[col_perm[i]] присваивается равным y[i], где col_perm — вектор перестановок столбцов. В результате получается вектор x, являющийся решением исходной системы. После нахождения решения вычисляется невязка — вектор r = A·x - b, характеризующий точность полученного решения. Для количественной оценки точности используются две нормы невязки: евклидова норма L2, вычисляемая как квадратный корень из суммы квадратов компонент вектора невязки, и максимальная норма L∞, представляющая собой максимальное по модулю значение среди всех компонент невязки. Малая невязка свидетельствует о том, что найденное решение хорошо удовлетворяет исходной системе уравнений. Основными преимуществами модифицированного метода Гаусса с полным выбором ведущего элемента являются повышенная численная устойчивость за счёт минимизации ошибок округления, возможность решения систем с плохо обусловленными матрицами (в определённых пределах), а также автоматическое выявление вырожденных систем, когда максимальный элемент в подматрице оказывается меньше заданного порога чувствительности. К недостаткам метода можно отнести более высокую вычислительную сложность — $O(n^3)$ операций, как и в классическом методе, но с дополнительными затратами на поиск максимального элемента во всей подматрице, что увеличивает константу времени выполнения примерно в два раза. Тем не менее, для большинства практических задач повышение устойчивости оправдывает дополнительные вычислительные затраты.

Для проверки вычисления проводились рассчеты невязка (residual) — это вектор, характеризующий точность найденного решения системы линейных алгебраических уравнений.

Для системы вида:

$A \cdot x = b$

A — матрица коэффициентов размером n × n

x — вектор неизвестных размером n

b — вектор правой части размером n

Вектор невязки определяется как:

$r = A \cdot x - b$

где x — найденное численное решение.

Вычисление невязки в программе

В разработанной программе вычисление невязки выполняется в функции computeResidual по следующему алгоритму:

$r_i = \Sigma_{j=1}^{n} a_{ij} \cdot x_j - b_i, i = 1,2, \dots, n$

Евклидова норма $\|\boldsymbol{r}\|_2$

$$\|r\|_2 = \sqrt{(r_1^2 + r_2^2 + \cdots + r_n^2)} = \sqrt{\left(\Sigma_{i=1}^{n} r_i^2\right)}$$

Данная величина показывает среднеквадратичную ошибку по всем уравнениям, а также она чувствительна к большим отклонениям

Максимальная норма $\|\boldsymbol{r}\|_\infty$

$\|r\|_\infty = max(|r_1|, |r_2|, \dots, |r_\mathrm{n}) = max_i |r_i|$

Показывает максимальное отклонение в одном уравнении и даёт строгую оценку точности.

Метод простых итераций (Якоби)

Итерационный метод, основанный на приведении системы к виду

$x = Bx + c$, где $B = -D^{-1}(L + U)$, $c = D^{-1}b$, D – диагональная часть матрицы A, а L, U – содержат верхнюю и нижнюю треугольные части матрицы.

Формула пересчёта

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j \neq i \\ j=1}}^{n} a_{ij}x_j^{(k)}\right), i = \overline{1,n}$$

Теперь найдём элементы B:

- Для $i \neq j$:$(L+U)_{ij} = a_{ij}$,так как L и U содержат в се недиагональные элементы. Умножение на D^{-1} слева даёт $\left(-D^{-1}(L+U)_{ij}\right) = -\frac{a_{ij}}{a_{ii}}$.
- Для $i = j$: $(L+U)_{ii} = 0$, поэтому $b_{ii} = 0$

Поэтому для оценки сходимости метода по норме $\|B\|_\infty$ (максимальная сумма модулей элементов строки) достаточно для каждой строки i вычислить

$$\sum_{j \neq i} |b_{ij}| = \sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|}$$

и взять максимум по всем строкам. Если эта норма меньше единицы, сходимость метода гарантирована. Если норма близка к единице, сходимость будет медленной; если норма больше или равна единице, метод может расходиться.

$$\|B\|_\infty = max\left(\sum_{j=1}^{n} |a_{ij}|\right), i = \overline{1,n}$$

Условие сходимости: спектральный радиус матрицы B должен быть меньше 1. Для практической оценки используется норма $\|B\|_\infty$. Если $\|B\|_\infty < 1$, сходимость гарантирована; если близка к 1, сходимость медленная; если ≥ 1, метод может расходиться.

В программе реализован критерий остановки по достижении заданной точности tol = 1e-12 по евклидовой норме изменения решения.

Выполнение работы

Метод гаусса с полным выбором

bool gauss_complete_pivoting(vector<vector<long double>>& A, vector<long double>& x) — принимает расширенную матрицу A размера n×(n+1) и ссылку на вектор x для записи решения, возвращая true при успешном решении и false в случае обнаружения вырожденной матрицы. Внутри этой функции сначала инициализируются векторы перестановок строк row_perm и столбцов col_perm. Затем выполняется прямой ход: для каждого шага k от 0 до n-1 происходит поиск максимального по модулю элемента в подматрице A[k..n-1][k..n-1], если найденное значение оказывается меньше порога 1e-14, функция возвращает false (матрица вырождена); после этого производятся перестановки строк и столбцов с обновлением векторов перестановок, и для всех строк ниже k вычисляется множитель factor = A[i][k]/A[k][k], после чего из текущей строки вычитается ведущая строка, умноженная на factor, что обнуляет элементы под диагональю. Далее выполняется обратный ход: для i от n-1 до 0 вычисляется сумма sum = Σ_{j=i+1}^{n-1} A[i][j]·y[j], затем y[i] = (A[i][n] - sum)/A[i][i], и в конце происходит восстановление исходного порядка неизвестных путём присваивания x[col_perm[i]] = y[i]. Для оценки точности полученного решения служит функция

void computeResidual(const vector<vector<long double>>& A_orig, const vector<long double>& b_orig, const vector<long double>& x, long double& norm_l2, long double& norm_linf), которая принимает исходную матрицу, вектор правой части, найденное решение и ссылки для записи двух норм невязки: евклидовой norm_l2 (квадратный корень из суммы квадратов невязок) и максимальной norm_linf (наибольшее по модулю значение среди всех компонент невязки). Для каждой строки i вычисляется сумма Σ A_orig[i][j]·x[j], находится невязка как разность этой суммы и b_orig[i], затем накапливается сумма квадратов для L2 и отслеживается максимум абсолютных значений для L∞, после цикла извлекается квадратный корень. Для удобства интеграции с библиотекой Eigen и основным меню программы создана функция-обёртка

bool solveGauss(const MatrixXd& A_eigen, const VectorXd& b_eigen, vector<long double>& x, double& elapsed, long double& l2, long double& linf), которая преобразует входные данные из формата Eigen в расширенную матрицу vector<vector<long double>>, замеряет время выполнения gauss_complete_pivoting с помощью std::chrono, затем при успехе вычисляет невязку через computeResidual и возвращает результат, заполняя переданные по ссылке время elapsed (в секундах) и нормы невязки l2 и linf. Помимо этих функций, в программе реализован ряд генераторов тестовых систем: generateRandomSystem создаёт случайную матрицу с элементами из диапазона [-10,10] и эталонное решение, generateDiagonallyDominantSystem формирует диагонально преобладающую матрицу (каждый диагональный элемент гарантированно больше суммы модулей остальных элементов строки), generateHilbertSystem генерирует плохо обусловленную матрицу Гильберта с элементами 1/(i+j+1), а generateSingularSystem создаёт вырожденную матрицу путём линейной комбинации строк. Все генераторы (кроме вырожденного) также создают эталонное решение x* и вычисляют правую часть как b = A·x*, что позволяет впоследствии вычислять истинную погрешность ||x - x*||. Основная функция main организует интерактивное меню, в котором пользователь может выбрать тип системы, запустить решение методом Гаусса (пункт 3), вывести решение, невязку и время, а также сравнить результат с эталоном и с другими методами (Eigen и Якоби). Все операции с плавающей точкой в реализации метода Гаусса используют тип long double для повышения точности, а при вычислении невязки и сравнении решений применяются научный формат вывода (scientific) с точностью до 12 знаков. Благодаря такому модульному построению код легко тестировать, модифицировать и расширять, а использование полного выбора ведущего элемента обеспечивает численную устойчивость метода даже для систем с плохой обусловленностью (в пределах, допустимых для арифметики с плавающей точкой).

Метод библиотеки Eigen

bool solveEigen(const MatrixXd& A, const VectorXd& b, VectorXd& x, double& elapsed, double& l2, double& linf), которая обеспечивает интерфейс между основным меню программы и высокооптимизированными алгоритмами Eigen. Функция принимает следующие параметры: матрицу коэффициентов A типа Eigen::MatrixXd (динамическая матрица двойной точности), вектор правой части b типа Eigen::VectorXd, ссылку на вектор x для записи решения, а также ссылки для возврата времени выполнения elapsed (в секундах), евклидовой нормы невязки l2 и максимальной нормы невязки linf. В начале работы функции с помощью std::chrono::high_resolution_clock фиксируется начальный момент времени. Затем создаётся объект FullPivLU<MatrixXd> lu(A), который выполняет LU-разложение матрицы A с полным выбором ведущего элемента (то есть с перестановками как строк, так и столбцов) — это наиболее устойчивый вариант LU-разложения, предоставляемый Eigen. После этого вызывается метод lu.isInvertible(), который проверяет, является ли матрица невырожденной (обратимой). Если матрица невырождена, то решение находится вызовом lu.solve(b), который решает систему A·x = b с использованием выполненного LU-разложения. Если же матрица вырождена, функция возвращает false, при этом время выполнения обнуляется, а нормы невязки устанавливаются в ноль. После получения решения фиксируется конечный момент времени, вычисляется продолжительность elapsed = duration.count(). Далее вычисляется вектор невязки residual = A·x - b, после чего его евклидова норма находится вызовом метода .norm(), а максимальная норма — вызовом .lpNorm<Infinity>() (что соответствует максимуму абсолютных значений компонент). Полученные значения норм невязки и время выполнения возвращаются через ссылочные параметры, а само решение — через ссылку x. В случае успешного решения функция возвращает true. Важно отметить, что в отличие от собственной реализации метода Гаусса, библиотечная функция не прерывает выполнение при вырожденной матрице, а предоставляет пользователю информацию о

вырожденности через возвращаемое значение, позволяя при необходимости переключиться на альтернативные методы (например, на SVD-разложение). В основной программе вызов solveEigen осуществляется из меню (пункт 2), после чего полученное решение выводится в консоль вместе с нормами невязки и временем решения, а также, если доступно эталонное решение, вычисляется и выводится погрешность ||x - x*|| в евклидовой и максимальной нормах. Для обеспечения максимальной производительности компиляция программы выполняется с ключами оптимизации -O2 или -O3, что позволяет Eigen эффективно использовать векторизованные инструкции процессора (SSE, AVX) и достигать высокой скорости решения даже для матриц размером 1000×1000 и более. Таким образом, функция solveEigen служит удобным мостом между пользовательским интерфейсом и мощными численными алгоритмами библиотеки Eigen, обеспечивая как высокую точность, так и высокую производительность.

Метод простых итераций (Якоби)

В разработанной программе для решения систем линейных алгебраических уравнений итерационным методом Якоби (методом простых итераций) реализованы две основные функции:

double computeJacobiNorm(const MatrixXd& A)

bool jacobi(const MatrixXd& A, const VectorXd& b, VectorXd& x, double& elapsed, double& l2, double& linf, int maxIter, double tol), а также используется вспомогательная функция генерации диагонально преобладающих систем для демонстрации эффективности метода.

Функция **double computeJacobiNorm(const MatrixXd& A)** вычисляет норму матрицы итераций Якоби B = -D^{-1}(L+U) по бесконечности (строчной норме). Для каждой строки i вычисляется сумма sum = Σ_{j≠i} |a_{ij}| / |a_{ii}|, и норма определяется как максимум таких сумм по всем строкам. Если какой-либо диагональный элемент оказывается близким к нулю (по модулю меньше 1e-15), функция возвращает заведомо большое число (1e20), сигнализирующее о неприменимости метода. Эта норма

является практическим критерием сходимости: если $\|B\|_\infty < 1$, сходимость метода гарантирована; если норма близка к единице, сходимость будет медленной; если норма больше или равна единице, метод может расходиться. Вычисленное значение выводится на экран перед началом итераций для информирования пользователя о прогнозируемой скорости сходимости.

Основная функция **bool jacobi(const MatrixXd& A, const VectorXd& b, VectorXd& x, double& elapsed, double& l2, double& linf, int maxIter, double tol)** реализует сам итерационный процесс. Она принимает матрицу коэффициентов A, вектор правой части b, ссылку на вектор x для записи решения, ссылки для возврата времени выполнения elapsed и норм невязки l2, linf, а также параметры maxIter (максимальное число итераций, по умолчанию 100000) и tol (требуемая точность, по умолчанию 1e-12). В начале работы функция проверяет, что все диагональные элементы матрицы не равны нулю (сравнение с 1e-15), и если это не так, выводит сообщение об ошибке и возвращает false. Затем вызывается computeJacobiNorm для оценки сходимости, и если норма оказывается больше или равной единице, выводится предупреждение о возможной расходимости или медленной сходимости. Далее инициализируется начальное приближение x нулевым вектором, создаётся вектор x_new для хранения следующего приближения, и фиксируется начальный момент времени с помощью std::chrono::high_resolution_clock. После этого запускается итерационный цикл, который продолжается до достижения максимального числа итераций. На каждой итерации для каждой переменной i от 0 до n-1 вычисляется сумма $sum = \Sigma_{j\neq i} A(i,j) * x(j)$, после чего новое значение переменной вычисляется по формуле x_new(i) = (b(i) - sum) / A(i,i). Важно отметить, что в классическом методе Якоби все новые значения вычисляются на основе значений предыдущей итерации, поэтому требуется отдельный вектор x_new. После вычисления всех компонент x_new вычисляется ошибка err = (x_new - x).norm() (евклидова норма изменения решения), затем вектор x обновляется присваиванием x = x_new. Если ошибка становится меньше

заданной точности tol, итерационный процесс останавливается досрочно, фиксируется конечное время, вычисляются невязка A·x - b и её нормы L2 и L∞, после чего функция возвращает true, а через ссылочные параметры возвращаются время выполнения и нормы невязки. Если максимальное число итераций достигнуто, а требуемая точность не обеспечена, функция также вычисляет невязку и время, но возвращает false, выводя сообщение о том, что метод не сошёлся.

Вывод

В ходе выполнения лабораторной работы были успешно реализованы и исследованы три различных подхода к решению систем линейных алгебраических уравнений: прямой метод Гаусса с полным выбором ведущего элемента, итерационный метод Якоби (простых итераций) и библиотечный метод Eigen, основанный на LU-разложении. Разработанная программа на языке C++ позволяет генерировать тестовые системы различных типов (случайные, диагонально преобладающие, Гильберта, вырожденные), решать их тремя указанными способами, сравнивать полученные решения с эталонным (которое генерируется вместе с системой) и оценивать точность через нормы невязки L2 и L∞, а также через нормы погрешности относительно точного решения. Проведённые вычислительные эксперименты показали, что для хорошо обусловленных систем (диагонально преобладающих или случайных с умеренным числом обусловленности) все три метода дают решения с высокой точностью (порядка 10^{-14}), при этом метод Якоби сходится за небольшое число итераций (как правило, менее 50), а прямые методы (Гаусс и Eigen) решают систему за доли секунды даже для матриц размером 1000×1000. Для плохо обусловленных систем, в частности для матрицы Гильберта при $n \geq 12$, ни один из методов не способен получить решение приемлемой точности в арифметике double из-за экспоненциального роста числа обусловленности, а метод Якоби расходится, так как норма матрицы итераций превышает единицу. Для вырожденных систем метод Гаусса корректно определяет

вырожденность по порогу ведущего элемента и выводит сообщение об ошибке, тогда как библиотека Eigen продолжает вычисления, возвращая одно из бесконечного множества решений с нулевой невязкой, что может ввести пользователя в заблуждение без дополнительного анализа ранга матрицы. В программу был добавлен диагностический механизм для метода Якоби: вычисление и вывод нормы матрицы итераций $\|B\|_\infty$ позволяет пользователю заранее оценить сходимость метода — если норма меньше единицы, сходимость гарантирована и будет тем быстрее, чем меньше норма; если норма близка к единице, сходимость будет медленной; если норма больше или равна единице, метод может расходиться. Экспериментально подтверждено, что для диагонально преобладающих матриц норма $\|B\|_\infty$ всегда меньше единицы, поэтому метод Якоби эффективен, а для матрицы Гильберта норма значительно превышает единицу, что приводит к расходимости. Таким образом, цели лабораторной работы достигнуты: реализованы и протестированы прямые и итерационные алгоритмы решения СЛАУ, выявлены их преимущества и недостатки, определены границы применимости в зависимости от обусловленности матрицы и её диагональной доминантности, а также проведено сравнение с эталонной библиотекой Eigen, подтверждающее корректность собственных реализаций.